SD - Assignment 1

Analysis and Design Document

Subject: Software Design
Student: Butuza Raisa Briana

1. Requirements Analysis

1.1 Assignment Specification

The objective of this project is to develop a full-stack application for managing an **event reservation system**.

The application allows:

- Users to browse events and reserve seats
- Organizers to create and manage events

Each user has a defined role (**USER** or **ORGANIZER**) which determines the actions they are allowed to perform.

The system is implemented using:

- **Spring Boot** for the backend
- **Angular** for the frontend

The application ensures:

- Data integrity
- Input validation
- Business rule enforcement
- Clear error handling

1.2 Functional Requirements

The application has the following functional requirements:

- **User Management**
  - Users can register, log in, and manage their profile
- **Role Management**
  - Support role-based access control (USER / ORGANIZER)
- **Event Management**
  - Organizers can:
    - Create events
    - View events
    - Update events
    - Delete events
- **Reservation Management**
  - Users can:
    - Browse available events

        - Reserve seats
- **Input Validation**
    - Validate:
        - Name format
        - Email format
        - Password strength
        - Event details (title, date, location, capacity)
- **Duplicate Prevention**
    - Prevent multiple accounts with the same email
- **Authorization Rules**
    - Only organizers can manage events
- **Error Handling**
    - Display clear and user-friendly error messages

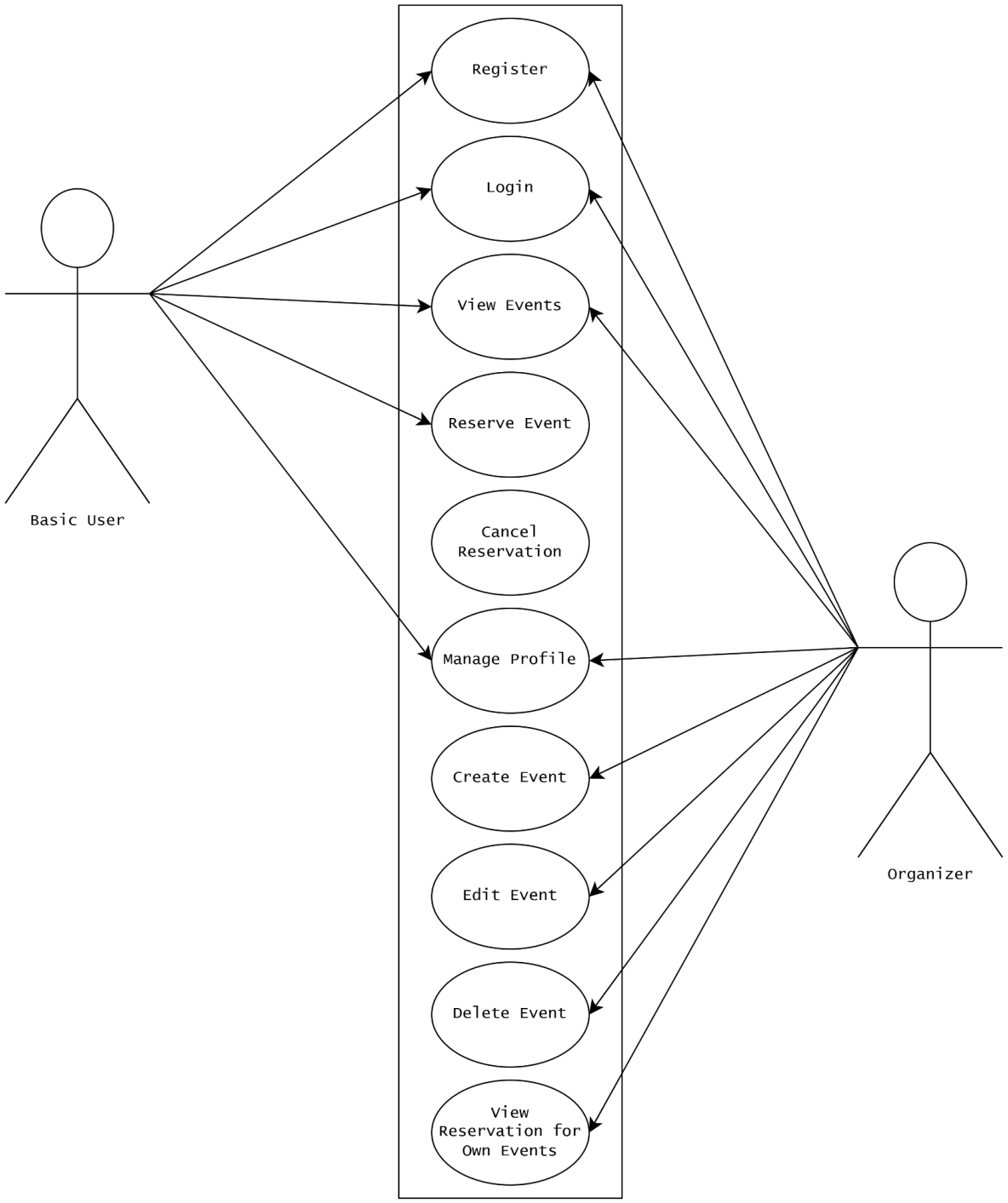
1.3 Non-functional Requirements

The application has the following non-functional requirements:

The application has the following non-functional requirements:

- **Layered Architecture**
    - Controller layer
    - Service layer
    - Repository layer
- **ORM Integration**
    - Use **Spring Data JPA**
- **Dependency Injection**
    - Use Spring DI container
- **Global Exception Handling**
    - Centralized error handling with meaningful HTTP responses
- **Scalability & Maintainability**
    - Modular and extensible design
- **Data Integrity**
    - Ensure consistency of reservations and event capacity
- **Security**
    - Authentication and role-based authorization

2. Use-Case Model



Organizer

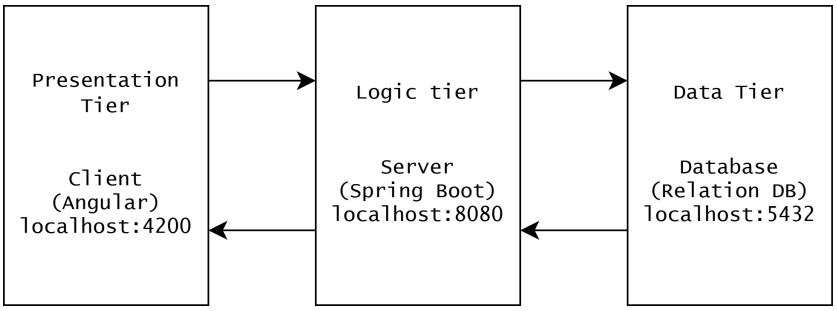
- Creates, edits, deletes events
- Views reservations

Basic User

- Browser events
- Makes reservations

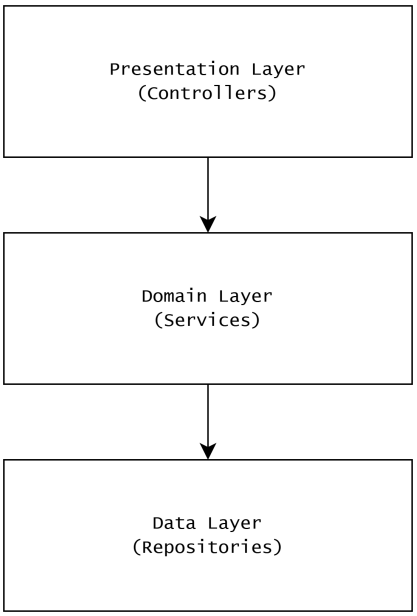
3. System Architecture Design

3.1 Architectural Pattern Description



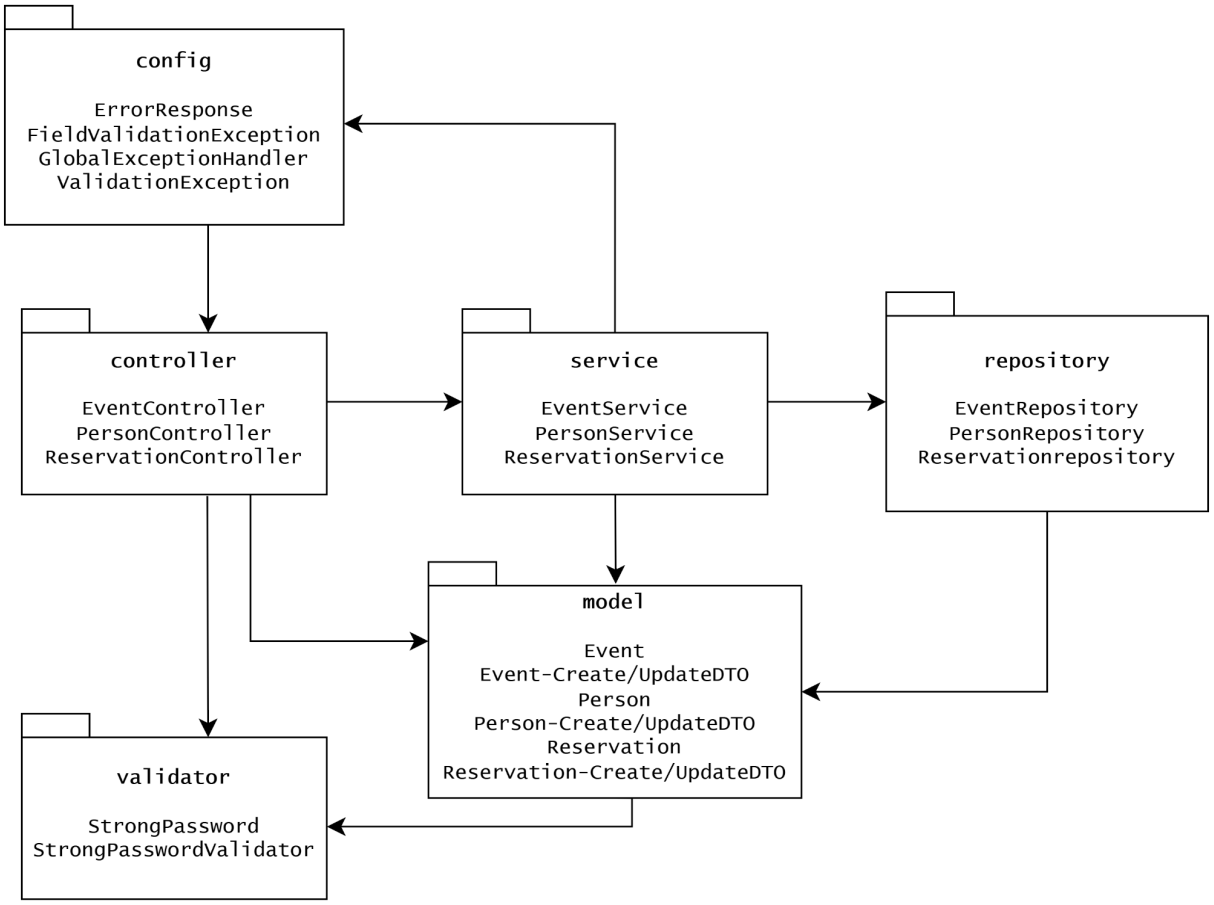
- **Presentation Tier (Client):** Implemented as a dynamic frontend using Angular. It handles user interactions, sends requests to the backend, and displays clear, user-friendly error messages when a request fails due to validation or server errors.
- **Logic Tier (Server):** Implemented as a robust backend with Spring Boot. This tier manages the core application logic, relying on a Dependency Injection (DI) container to manage component life cycles. It enforces strict input validation and features a centralized Global Exception Handler to capture business logic failures and map them to appropriate HTTP responses.
- **Data Tier (Database):** Uses a relational database integrated with Spring Data JPA (ORM) to handle all database interactions and entity mapping

3.2 Layer Architecture

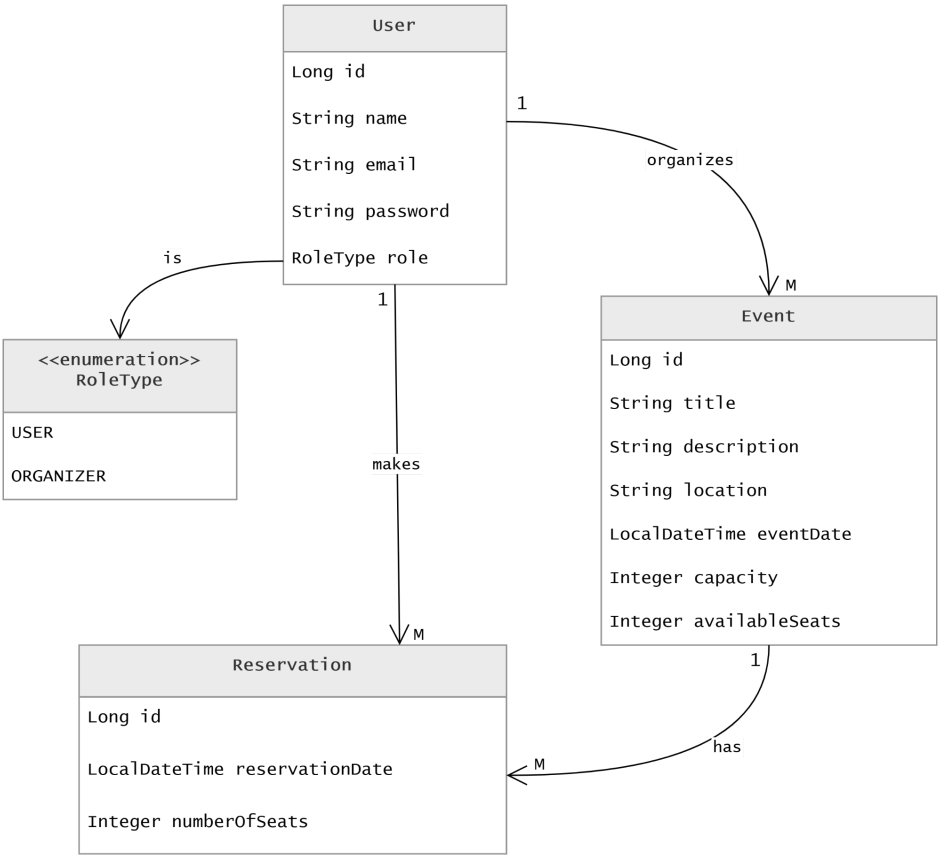


- **Presentation Layer (Controllers):** Acts as the entry point for REST API requests. It is responsible for handling routing, enforcing initial input validation using @Valid, and working alongside the Global Exception Handler to return consistent error responses.
- **Domain Layer (Services):** Holds the business logic. It ensures data integrity and prevents edge cases, such as creating multiple accounts with the same email address or assigning invalid relationships.
- **Data Layer (Repositories):** Provides data access. Using Spring Data JPA, it abstracts standard SQL operations into simple interfaces for seamless database manipulation and entity mapping.

3.3 Package Diagram



3.4 Class Model



Person ↔ Event (One-to-Many)

A Person with the role ORGANIZER can create many Events, but each Event has exactly one organizer. This is a one-to-many relationship mapped via the `organizer_id` foreign key in the Event table. In JPA, the Event entity holds a `@ManyToOne` reference to Person.

Person ↔ Reservation (One-to-Many)

A single Person (with role USER) can have many Reservations — one per event they want to attend. Each Reservation belongs to exactly one Person. This is again a one-to-many, mapped by the `person_id` foreign key in the Reservation table.

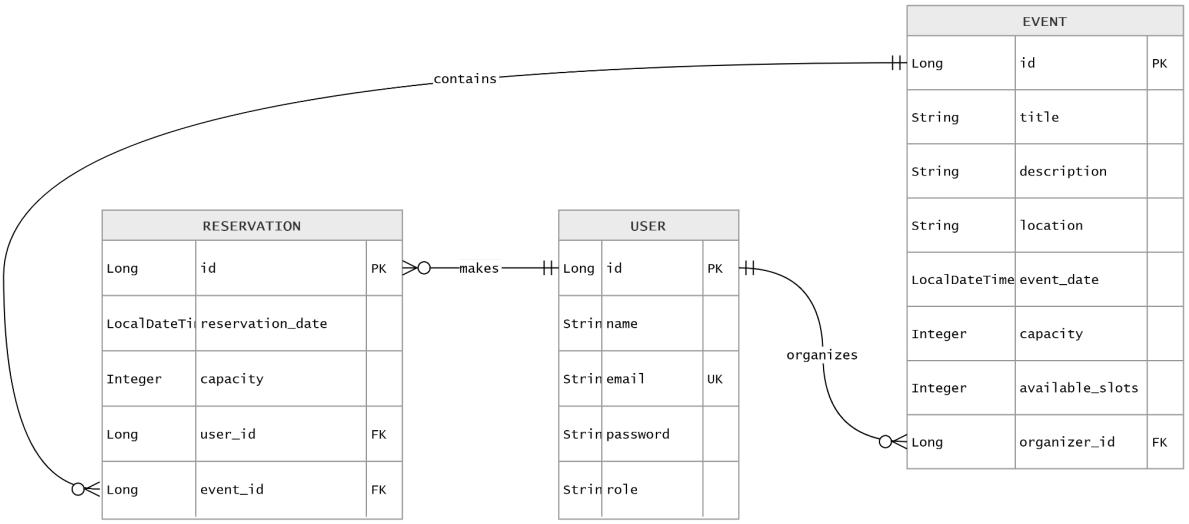
Event ↔ Reservation (One-to-Many)

A single Event can have many Reservations from different users. Each Reservation is tied to exactly one Event via the `event_id` foreign key. This is a one-to-many relationship.

Person ↔ Event (Many-to-Many through Reservation)

While there is no direct many-to-many mapping in the code, Person and Event are effectively connected in a many-to-many fashion through the Reservation join entity. A Person can reserve spots at many Events, and an Event can have many Persons reserving spots. The Reservation table acts as the associative/junction table, enriched with extra fields: `spots_reserved` and `status` (PENDING, ACCEPTED, DECLINED).

3.5 Data Model



User

- id - unique identifier
- name - full name of the user
- email - unique email address
- password - encrypted password
- role - role of the user (USER or ORGANIZER)

Event

- id - unique identifier
- title - name of the event
- description - event details
- location - place where the event will take place
- eventDate - scheduled date and time
- capacity - maximum number of seats
- availableSeats - remaining number of seats
- organizer - the user who created the event

Reservation

- id - unique identifier
- reservationDate - date and time when the reservation was made
- numberOfSlots - number of reserved seats
- user - the user who made the reservation
- event - the reserved event